

The ABBA Numerical-Method Family

Duplicated phase space, projection, composition, and configuration axes

1 Purpose and model boundary

The ABBA model is a family of integrators built from the same explicit A – B – B – A map on two copies of a physical state. A public class selects the order and projection placement; configuration controls select the projection equations, nonlinear solver, state extension, and optional physical energy tracking. These choices are configurations of a model, not additional model identities.

The family has five public methods:

- `ABBA2Midpoint` and `ABBA2Implicit`;
- `ABBA4Implicit` and `ABBA4ImplicitSingleProjection`; and
- `ABBA6Implicit`.

Their focused theory documents live in the sibling model directories.

2 Duplicated-space ABBA map

Let $\dot{z} = f(t, z)$ with $z \in \mathbb{R}^m$. Duplicate the state as (u, v) and set $s = h/2$. One signed endpoint-time step beginning at t is

$$u_1 = u_0 + sf(t, v_0), \quad v_1 = v_0 + sf(t, u_1), \quad (1)$$

$$v_f = v_1 + sf(t + h, u_1), \quad u_f = u_1 + sf(t + h, v_f). \quad (2)$$

We denote this map by $\Psi_{h,t}(u_0, v_0) = (u_f, v_f)$. Its reverse is $\Psi_{-h,t+h}$, so the endpoint-time convention is essential for non-autonomous reversibility. When the split system is Hamiltonian, each line is a shear and the composition is symplectic on the duplicated space.

With

$$E = \begin{pmatrix} I \\ I \end{pmatrix}, \quad P = \frac{1}{2} \begin{pmatrix} I & I \end{pmatrix}, \quad G = \begin{pmatrix} I & -I \end{pmatrix}, \quad N = G^T,$$

Ez embeds the physical diagonal, P takes its arithmetic mean, and $G(u, v) = u - v$ measures copy separation.

3 Projection choices

The midpoint method accepts $P\Psi_{h,t}(Ez)$ directly. The implicit methods use Hairer's symmetric projection. In the reduced formulation a multiplier $\mu \in \mathbb{R}^m$ displaces the input and corrects the output:

$$\hat{Y} = Ez + N\mu, \quad M = \Psi_{h,t}(\hat{Y}), \quad (3)$$

$$r(\mu) = GM + 2\mu = 0, \quad z^+ = P(M + N\mu). \quad (4)$$

If $J_\Psi = D\Psi_{h,t}(\hat{Y})$, Newton uses

$$D_\mu r = GJ_\Psi N + 2I.$$

The simultaneous formulation solves for the corrected doubled output Y and μ together:

$$F(Y, \mu) = \begin{pmatrix} Y - N\mu - \Psi_{h,t}(Ez + N\mu) \\ GY \end{pmatrix} = 0.$$

Eliminating Y gives the reduced equation. Thus both formulations define the same ideal projected map when they converge to the same root. Newton uses the exact residual Jacobian; good Broyden replaces repeated Jacobian assembly by rank-one secant updates without changing the defining residual.

4 Canonical configuration space

Axis	Canonical values and meaning
Projection formulation	<code>reduced_multiplier</code> ; <code>simultaneous_state_multiplier</code> .
Nonlinear solver	<code>newton</code> ; <code>broyden</code> .
State extension	<code>physical</code> ; <code>fully_extended</code> .
Energy tracking	<code>False</code> ; <code>True</code> . Fully extended execution normalizes this value to <code>True</code> .

The state-extension values are `physical` and `fully_extended`; physical execution may disable or enable energy tracking, while fully extended execution always enables it. Thus the three canonical normalized strategies are `(physical, False)`, `(physical, True)`, and `(fully_extended, True)`. Each implicit class admits $2 \times 2 \times 3 = 12$ canonical configurations. `ABBA2Midpoint` has no nonlinear equations and supports the same three strategies. The family therefore contains $4 \times 12 + 3 = 51$ canonical public configurations while retaining five method classes.

For one guiding-centre particle, the state dimensions are

Extension and tracking	accepted	split	reduced unknown	simultaneous unknown
physical, off	2	4	2	6
physical, on	2	4	2	6
fully extended, on	4	8	4	12

The simultaneous vector is nonlinear-solver workspace, not a trajectory state. Physical energy tracking reuses the accepted ABBA stages to advance one auxiliary normalized momentum $\kappa = k/2$ per particle. The update is triangular: it does not feed back into z and does not change the accepted, split, observer, or nonlinear-workspace dimensions. Fully extended mode instead duplicates $Z = (z, t, k)$ and projects the complete autonomous state.

5 Order by symmetric composition

The projected ABBA2 map is a symmetric method of order two. Higher order is obtained by palindromic signed compositions. ABBA4 uses the triple jump (γ, δ, γ) with

$$\gamma = \frac{1}{2 - \sqrt[3]{2}}, \quad \delta = -\frac{\sqrt[3]{2}}{2 - \sqrt[3]{2}}, \quad 2\gamma + \delta = 1, \quad 2\gamma^3 + \delta^3 = 0.$$

ABBA6 uses seven Yoshida coefficients whose first, third, and fifth odd moments have the required cancellations. Negative coefficients are intrinsic to these real compositions, so the dynamics must be defined at all internal signed times.

6 Projection placement is part of the model

Method	ABBA maps	projection policy
ABBA2Midpoint	1	arithmetic mean
ABBA2Implicit	1	one symmetric projection
ABBA4Implicit	3	project after every signed map
ABBA4ImplicitSingleProjection	3	project around the complete triple jump
ABBA6Implicit	7	project after every signed map

The two fourth-order policies define different numerical maps even though they share the same coefficients.

7 Finite solves and geometric claims

Symmetry, formal order, and symplecticity concern the ideal map obtained from the exact projection root. A finite stopping tolerance and floating-point arithmetic perturb that map. Consequently, the implementation records iterations, residual evaluations, final residual norms, tolerances, and projection multipliers. Step observers expose the accepted map so measured Jacobian defects can be distinguished from ideal algebraic properties.

8 Implementation correspondence

The common stage kernel is in `src/simulation/methods/abba/_core.py`; the two projection formulations are in `_projection_reduced.py` and `_projection_simultaneous.py`; configuration validation is in `_configuration.py`; and the five public classes are implemented by the corresponding `order*.py` modules. The architecture document in `../simulation/` is the authoritative runtime inventory.